

Instituto Tecnológico y de Estudios Superiores de Monterrey

Detección de Plagio

Diego Reséndiz Fernández A01708017

Félix Javier Rojas Gallardo A01201946

Jesús Ramírez Delgado A01274723

Desarrollo de Aplicaciones Avanzadas de Ciencias Computacionales

Abstract

En los últimos años, el uso de herramientas de inteligencia artificial para el desarrollo de software ha crecido de manera acelerada, abriendo nuevas posibilidades para la generación, modificación y optimización de código. Sin embargo, este crecimiento también ha generado retos importantes relacionados con la calidad, seguridad y autoría del software. Uno de los problemas más relevantes es el plagio de código asistido por IA, ya que estas herramientas pueden modificar, reestructurar u ofuscar soluciones existentes, dificultando la identificación de similitudes entre programas.

Este problema no solo afecta el ámbito académico, sino también el desarrollo profesional de software. La práctica conocida como *vibe coding*, cuando se utiliza sin una comprensión adecuada del código generado, puede introducir errores funcionales, vulnerabilidades de seguridad y deuda técnica. En consecuencia, sistemas aparentemente funcionales pueden volverse difíciles de mantener, corregir o escalar.

Ante esta problemática, el presente proyecto propone el desarrollo de un modelo basado en una red neuronal multicapa —MLP— para detectar posibles casos de plagio en código Python. El modelo utiliza características léxicas, estructurales y semánticas extraídas de pares de código, con el objetivo de estimar la probabilidad de similitud sospechosa o plagio. De esta manera, se busca construir una herramienta auxiliar que aporte evidencia técnica para apoyar la revisión y análisis de autoría de código

Índice

1.0 Introducción.....	3
1.1 Objetivos.....	4
2.0 Marco teórico.....	4
3.0 Dataset.....	8
3.1 Origen del dataset.....	8
3.2 Descripción.....	8
3.4 Matriz de confusión.....	10
3.5 Features.....	10
3.5.1 Métricas de Similitud Léxica y Textual.....	11
3.5.2 Análisis Estadístico y de Estilo.....	12
3.5.3 Diferenciales de Atributos (Métricas Relativas).....	12
4.0 Preprocesado de los datos.....	13
4.1 Corrección balance de clases.....	13
4.2 Obtención de features.....	13
4.3 Split de datos.....	14
4.4 Set de validación.....	14
5.0 Modelado.....	15
5.1 Regresión logística.....	15
5.1.1 Descripción.....	15
5.2 Multi Layer Perceptron Version 1.....	15
5.2.1 Descripción.....	15
5.2.2 Configuración y parámetros.....	15
5.2.3 Arquitectura.....	16

5.3 Multi Layer Perceptron Version 2.....	16
5.3.1 Descripción.....	16
5.3.2 Cambios y/o mejoras.....	17
5.3 Multi Layer Perceptron Version 3.....	17
5.3.1 Descripción.....	17
5.3.2 Cambios y/o mejoras.....	18
5.3.4 Arquitectura.....	19
6.0 Resultados obtenidos.....	19
6.1 Métricas de evaluación.....	19
6.2 Resultados.....	20
6.3 Mejoras.....	22
7.0 Ventajas y Desventajas del Modelos.....	22
7.1 Regresión logística.....	22
7.1.1 Ventajas.....	22
7.1.2 Desventajas.....	22
7.2 MLP Version 1.....	23
7.2.1 Ventajas.....	23
7.2.2 Desventajas.....	23
7.3 MLP Version 2.....	23
7.3.1 Ventajas.....	23
7.3.2 Desventajas.....	23
7.4 MLP Version 3.....	23
7.4.1 Ventajas.....	24
7.4.2 Desventajas.....	24
7.4.3 Áreas de oportunidad.....	24
8.0 Conclusiones generales.....	25
Citas.....	26

1.0 Introducción

El crecimiento del uso de herramientas de inteligencia artificial en el desarrollo de software ha generado nuevos retos relacionados con la autoría, originalidad y calidad del código. En particular, el plagio de código se ha vuelto más difícil de detectar, ya que las herramientas generativas pueden modificar nombres de variables, reordenar estructuras, cambiar el estilo de programación o reescribir fragmentos sin alterar la lógica principal.

En este contexto, la detección de plagio no puede depender únicamente de la comparación textual directa, ya que dos programas pueden resolver el mismo problema con estructuras muy similares, o bien ocultar una copia mediante pequeñas modificaciones. Por esta razón, es necesario utilizar un enfoque que considere características léxicas, estructurales y semánticas del código.

Este proyecto se enfoca en el desarrollo de una herramienta auxiliar especializada en código Python, capaz de comparar pares de archivos y estimar la probabilidad de que exista plagio o similitud sospechosa. La herramienta no busca reemplazar la revisión humana, sino proporcionar evidencia

técnica que facilite el análisis y apoye la toma de decisiones en contextos académicos o de revisión de autoría.

1.1 Objetivos

Desarrollar una herramienta auxiliar para detectar posible plagio en código Python, capaz de superar mínimamente un baseline de regresión logística mediante iteraciones de modelos MLP con características léxicas, estructurales y semánticas, buscando una propuesta más robusta y diferente a los enfoques tradicionales de comparación textual.

2.0 Marco teórico

1. Deep Learning para detección de plagio

Abstract: Propone un sistema basado en deep learning que detecta plagio incluso con modificaciones semánticas, mejorando métodos tradicionales de matching. [4]

2. Plagiarism Detection Using Machine Learning (2024)

Abstract: Explora modelos ML (clasificación supervisada) para detectar similitud textual y patrones de plagio en múltiples dominios. [5]

3. AI-Generated Text Detection (2024)

Abstract: Compara regresión logística, SVM y Naive Bayes para detectar contenido generado por IA, destacando limitaciones y mejoras.[6]

4. NLP-based Plagiarism Detection

Abstract: Integra NLP para analizar estructura y relaciones semánticas, aumentando precisión frente a simple coincidencia de texto.[7]

5. Evaluación de detectores de IA (2023)

Abstract: Los detectores funcionan mejor con GPT-3.5 que con GPT-4 y presentan falsos positivos en textos humanos.[8]

6. Survey de métodos de plagio (2025)

Abstract: Clasifica tipos de plagio y analiza algoritmos modernos incluyendo IA, embeddings y análisis semántico.[9]

7. Detección de código generado por IA (2024)

Abstract: Detectores actuales fallan en generalización; usar embeddings de AST y métricas estáticas mejora resultados ($F1 \approx 82\%$).[10]

8. CodeMirage Benchmark (2025)

Abstract: Dataset multilenguaje para evaluar detectores de código IA; revela debilidades en modelos actuales. [11]

9. AI Detection via Rewriting (Raidar)

Abstract: Detecta IA midiendo cuánto cambia un texto al reescribirlo con otro modelo; mejora F1 hasta +29%. [12]

10. PADBen Benchmark (2025)

Abstract: Detectores fallan ante paraphrase attacks; dificultad clave es distinguir contenido “lavado” semánticamente. [13]

11. AI Code Detection using pseudo submissions

Abstract: Métodos como XGBoost y RoBERTa logran alta precisión, pero fallan con código generado por IA. [14]

12. Concepto general de plagio (base teórica)

Abstract: Define plagio como detección de duplicidad en texto o código mediante comparación estructural o semántica. [15]

Justificación de los artículos seleccionados

1. Detección de código generado por IA por Suh et al. [10]

Muestra que el estado del arte para detección de plagio con GPTSniffer está restringido en su mayor parte a código de Java. Las herramientas de análisis para código generado con IA (IAGC) no generalizan lo suficiente.

Propone una arquitectura en donde se tiene ML (Machine Learning) con features o métricas estáticas y AST para mejorar la detección de código generado por IA. Así juntando pistas como :

- ¿Cómo está escrito?
- ¿Cómo está organizado?
- ¿Cómo se ve por dentro?

El paper usa las siguientes features y nosotros retomamos algunas para nuestro análisis previo del código. (extraído de [10])

TABLE III: Collected Source Code Features

Features	Definitions
SumCyclomatic	Sum of cyclomatic complexity of all nested functions or methods.
AvgCountLineCode	Average number of lines containing source code for all nested functions or methods.
CountLineCodeDecl	Number of lines containing declarative source code
CountDeclFunction	Number of functions
MaxNesting	Maximum nesting level of (if, while, for, switch etc.)
CountLineBlank	Number of blank lines
Keywords	The ratio between the number of language keyword tokens to the number of total tokens
Operators in if, else, and while statements	The ratio between the number of operators in if, else, and while statements to the number of total tokens

Combinado con AST que relata la estructura del código y la relación de los elementos del código. Vectorizando toda la información recolectada para que el ML pudiera analizar patrones y aprenderlos para determinar si es IA y en qué porcentaje.

Consideramos importante obtener una pauta inicial sobre cómo comenzar el modelo, lo que debemos de tener en cuenta, por ello la utilidad de un paper con un acercamiento empírico. Muestra la importancia de comparar el AST aunado a elementos estáticos para complementar la detección.

2. PADBen Benchmark (2025) por Y. Zha, R. Min, and S. Sushmita [10]

El paper comienza con la idea de ataques de paráfrasis pero lo más importante aquí es la generación del dataset con el que se va a entrenar al modelo.

Establece que el dataset debe de tener distintas muestras:

1. Código humano
2. Código IA
3. Humano → modificado por IA
4. IA → modificado por IA

Todo esto debe ser generado por distintas IAs para que no se acostumbre al estilo de una sola IA sino que sea capaz de tener una generalización lo suficientemente buena para tener un porcentaje aceptable de detección de código generado por cualquier IA, no solo chat gtp, copilot, gemini, grok, deep seek, etc.

Escogimos este paper porque no basta con solo tener un montón de datos sino que debemos de considerar una *generalización de la detección de código* colocando distintos ejemplos dentro del dataset usando distintos métodos y distintas IAs para que así el modelo no se acostumbre a detectar de una sola IA sino que de todas así evitando cualquier confusión por el estilo de la IA.

3. Elkhata et al. [8] establecen que la detección de código generado por IA es más fácil cuando fue generado por modelos viejos y que los modelos al identificar cometen errores (falsos positivos). Propone el uso de un índice de tolerancia para evitar falsos positivos. Mientras más baja la tolerancia más estricto será el modelo. Pero da más falsos positivos; mientras más alta la tolerancia hay menos falsos positivos. Pero se escapa algunos códigos de IA. Por lo que se establece un índice de tolerancia del 0.7 - 0.8. Al mismo tiempo propone una clasificación por rangos (como se interpreta los datos)

0.0 ————— 0.4 ————— 0.6 ————— 0.8 ————— 1.0
Humano Duda Posible IA IA

La IA puede dar falsos positivos. Nos pareció importante poder reducir o ampliar el índice de tolerancia con una interpretación de rangos para determinar si es IA y en qué porcentaje lo es. Así el modelo es flexible a la hora de la detección. Obtenemos un margen de error más controlado que nos permite iterar sobre el modelo a utilizar.

4. CodeMirage Benchmark (2025) Guo et al. [8]

Establece que muchos modelos aciertan en los test de prueba pero fallan rotundamente cuando se les coloca en un entorno real. Esto se debe al problema de la generalización (El uso de patrones de desarrollo, por ejemplo) que hace más complicado la detección.

Establece que una condición ideal para su desempeño sería que el modelo y la detección se entrenen para un fin único. Por ejemplo: detectar IA en un código que tiene el objetivo de generar datos aleatorios. Al ser un objetivo específico, es más fácil detectar cualquier desviación que se pueda

atribuir a la IA. Pero al ser generalizado, la detección se vuelve más laxa porque se abarcan muchas cosas. Finalmente sugiere un dataset variado de muchos lenguajes, de muchas IAs y humanos.

Este paper lo seleccionamos porque nos dió una perspectiva acerca de la eficacia de la detección y sobre cómo el objetivo puede aumentar o reducir el accuracy de la detección.

5. Survey de métodos de plagio (2025) por Amirzhanov et al. [6]

Establece una base teórica sobre los tipos de plagio y algunos métodos de detección.

Tipo del paper	Equivalente en código	Features
Verbatim	Copiar código tal cual	Tokens
Paráfrasis	Mismo código con cambios	AST + estructura del código
Traducción	Cambiar lenguaje (Python → Java)	-
Idea	Misma lógica con otra implementación	Complejidad + comportamiento

Estos tipos de plagio, nos permite determinar técnicas para detectar si está hecho con IA y en qué porcentaje.

El paper habla de múltiples técnicas de procesamiento natural del lenguaje que podrían ser útiles para nuestro proyecto. No es redundancia sino que cada feature mide un aspecto distinto del código y al integrarlo todo, ya es muy difícil ofuscar el código. Es decir que se vuelve un modelo más robusto a pesar de no subir su complejidad.

Algunos ejemplos notorios:

1. Conteo del número de estructuras de control lógico (if/else) .
2. Conteo del número de bucles (for, while, do while, etc).
3. Se contarían las líneas de código.
4. Se contaría el número de variables.
5. Conteo de las estructuras de datos (listas, pilas, hashmaps, etc).
6. Conteo de las anidaciones de los ifs o for.
7. Sacar la profundidad del AST.
8. Conteo de los nodos del AST.
9. Conteo de las funciones creadas (def).
10. Conteo del número de Returns.
11. Usar un BOW o TF-IDF

6. AI-Generated Text Detection (2024), Moran Zeng [3]

Valida el uso de un modelo de regresión para la detección de código por IA. Exponiendo que un modelo de regresión es conceptualmente correcto ya que no es posible determinar si un código es 100% IA o no, sino que se debe de señalar la probabilidad de que el código sea IA para evitar falsos positivos. Además se compara con otros modelos y el resultado mostró que el modelo de regresión era

más simple y efectivo con un accuracy del 90%. Recomienda el uso de métricas como Accuracy, F1-score y ROC para medir el desempeño del modelo.

Escogimos este paper porque nos muestra algunas de las métricas que deberíamos de usar para medir el desempeño de la detección y nos da un marco para decir si nuestro modelo es bueno o es malo.

Cambió nuestra perspectiva inicial: teníamos planeado un modelo de clasificación de bloques de ejecución, pero es más sencillo y correcto hacerlo con regresión porque nos evita falsos positivos tajantes. Potencialmente obtendremos un porcentaje de probabilidad de que sea plagio o IA, haciendo el modelo flexible.

13. [Diverging Divergences: Examining Variants of Jensen Shannon Divergence for Corpus Comparison Tasks](#)

3.0 Dataset

3.1 Origen del dataset

El dataset utilizado fue obtenido de github, del repositorio [cheating_detector/DataSet at master · ehsankhani/cheating_detector](#) por el usuario Ehsankhani[16].

3.2 Descripción

El dataset son varios códigos en Python que usaron los alumnos para resolver una tarea que se les había pedido. El dataset marca con 1 aquellos archivos en relación a otros que no hayan sido plagio. Con 0 se marcan los que son plagio.

El dataset cuenta con 293 instancias.

3.3 Balance de clases

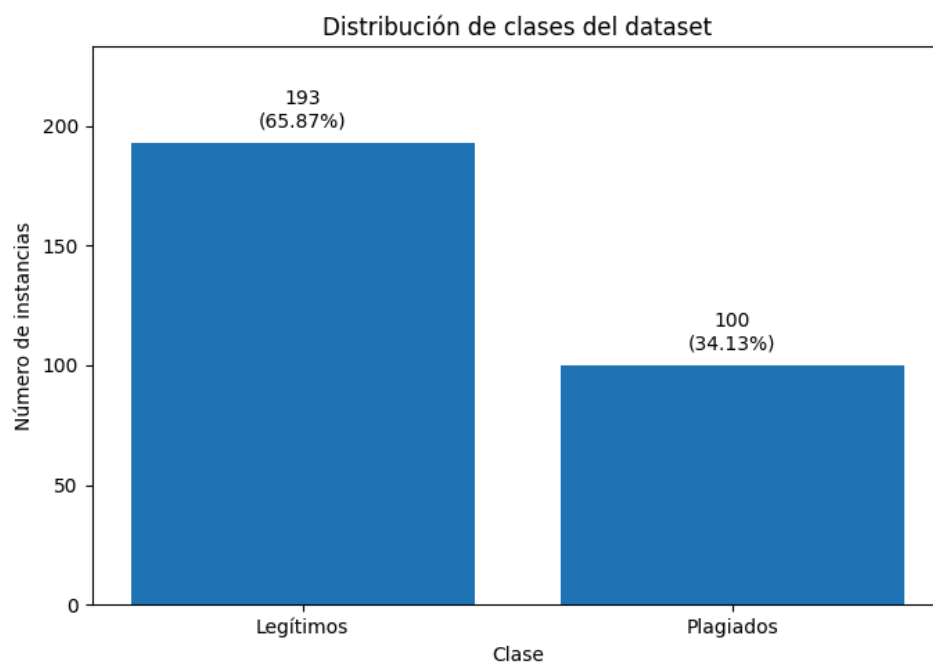


Figura 1.0: Distribución de la clases

Se puede observar un desbalance de clases teniendo una mayor concentración de textos legítimos que plagiados, debido a este desbalance, el modelo puede tender a clasificar con mayor facilidad los códigos legítimos, ya que tiene más ejemplos de esta clase durante el entrenamiento. Como consecuencia, podría presentar un mejor desempeño al identificar códigos no plagiados, pero un rendimiento menor al detectar casos de plagio.

3.4 Matriz de confusión

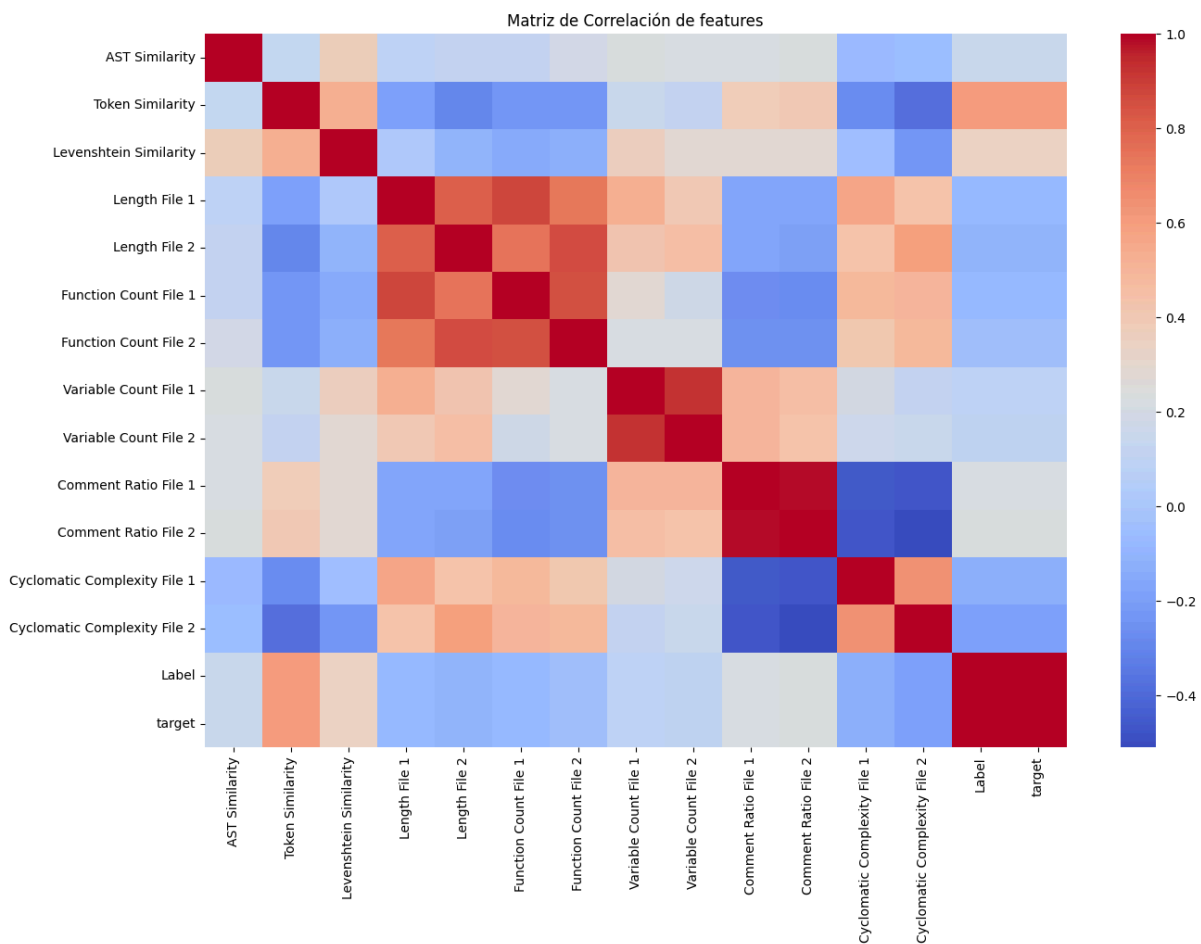


Figura 2.0: Matriz de correlación de features del modelo

Al interpretar la **matriz de confusión**, es importante prestar especial atención a los errores relacionados con la clase “plagiado”. En particular, los **falsos negativos** son críticos, ya que representan códigos plagiados que el modelo clasifica correctamente como legítimos. Este tipo de error reduce la capacidad del sistema para detectar plagio de forma efectiva.

3.5 Features

El proceso de selección de variables se basó inicialmente en una matriz de correlación, ésta nos permitió identificar qué métricas tenían un mayor peso estadístico respecto a la detección de plagio. Si bien los resultados mostraron que variables como *Token Similarity*, *Levenshtein*, *Comment Ratio* y *AST Similarity* eran las más influyentes.

La filosofía detrás de la selección de estas *features* es la relatividad: no analizamos archivos de forma aislada, sino que el modelo recibe las diferencias normalizadas y similitudes entre pares de códigos con las métricas que calculamos. Esto permite que la red neuronal identifique la distancia lógica (diferencia) y estructural entre dos códigos. Todo esto con el objetivo de subir la precisión del modelo tal y como se había plantado en el objetivo.

Se realizó un análisis PCA, donde se obtuvieron las siguientes métricas y su impacto:

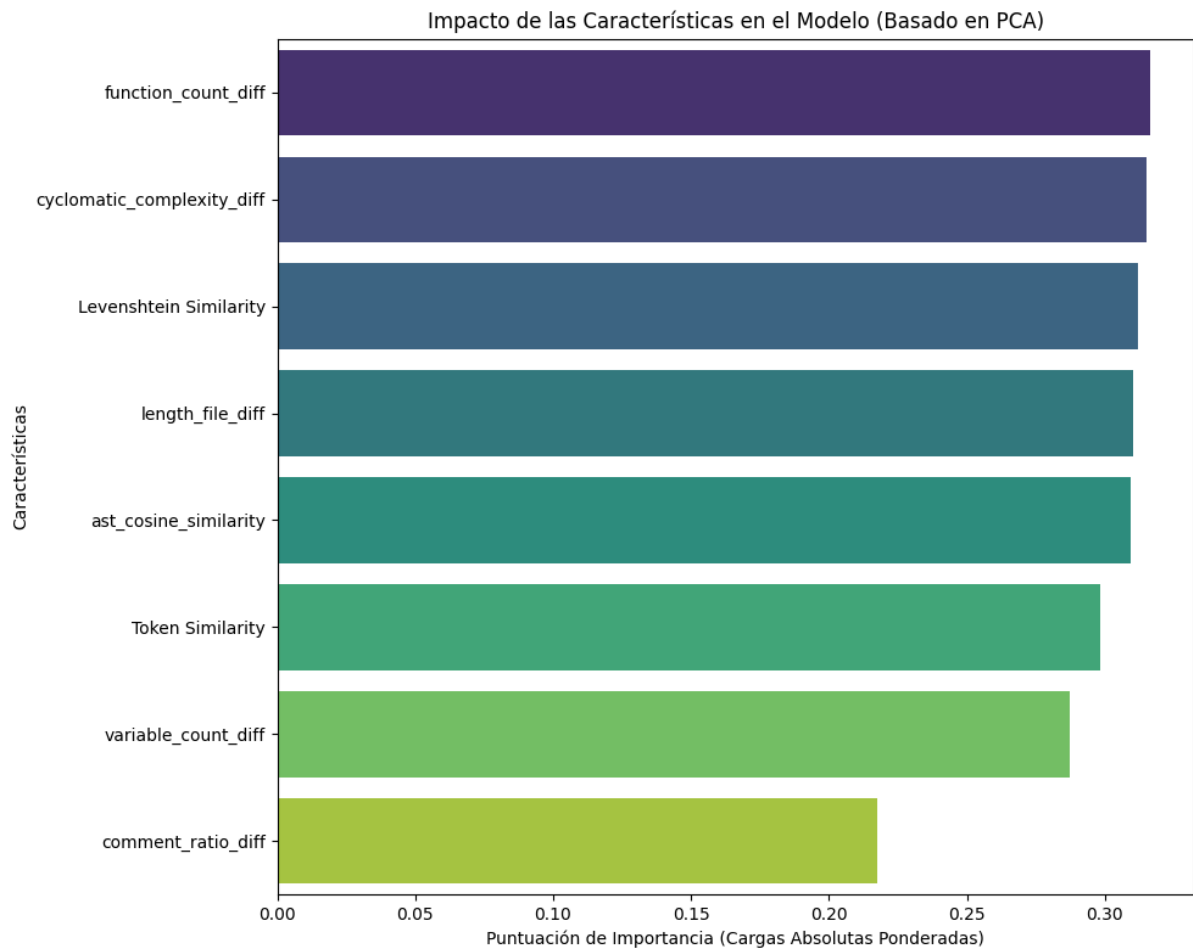


Figura 3.0: Matriz de impacto PCA de las features en el modelo

Las cuales:

3.5.1 Métricas de Similitud Léxica y Textual

Token Similarity:

- En qué se basa: En la estructura léxica. Antes de comparar, el código se convierte en *tokens* (palabras clave, operadores, identificadores).
- Cómo funciona: Ignora espacios en blanco y comentarios, comparando solo la lógica de los símbolos, se hace una intersección sobre la unión de los sets de tokens obtenidos (Similaridad de Jaccard)
- Aporte: Es vital porque detecta plagio incluso si el plagiador cambió los nombres de las variables o el formato del texto.

Levenshtein Similarity:

- En qué se basa: En la "distancia de edición" (cuántos cambios de un solo carácter son necesarios para transformar un código en otro).
- Cómo funciona: Compara las cadenas de texto crudo. Si dos códigos son casi idénticos pero cambian un par de variables, la distancia es mínima.
- Aporte: Detecta el plagio por "copia y pega" directo con modificaciones mínimas.

3.5.2 Análisis Estadístico y de Estilo

Coseno TF-IDF del AST con n-gram de 3.

- En qué se basa: Es una de las métricas mencionadas en Amirzhanov et al. [6] para mitigar la ofuscación con renombramiento de variables.
- Funcionamiento: Establece relaciones de los tokens en tríadas para capturar más contexto. Esto captura el estilo de escritura de código. Ejemplo: Comentario -> Función -> Variable. Se realiza la vectorización y se calculan las diferencia coseno de los vectores.
- Aporte: Estilo y estructura de código e Identifica si la lógica subyacente es la misma, aunque se han reordenado algunas partes del código.

3.5.3 Diferenciales de Atributos (Métricas Relativas)

Variable Count Difference: (variable_count_diff):

- En qué se basa: En la cantidad total de identificadores o variables declaradas y utilizadas dentro del código.
- Cómo funciona: El modelo extrae el número de variables únicas de cada archivo y calcula la diferencia absoluta entre ambos. Al igual que el resto de nuestras métricas, se trata de un valor relativo que mide la discrepancia en la densidad de datos del programa.
- Aporte: Esta métrica es fundamental para detectar la ofuscación por renombrado. Un plagiador puede cambiar todos los nombres de las variables (por ejemplo, cambiar suma_total por x), pero es muy inusual que agregue o elimine variables de la lógica central, ya que esto podría alterar el funcionamiento del código.

Ratio de comentarios (Comment_ratio_diff):

- En qué se basa: En la proporción de comentarios vs. código.
- Cómo funciona: Comparar qué tanto espacio ocupan las explicaciones.
- Aporte: El plagiador suele borrar comentarios o mantener la misma proporción. Una diferencia de cero aquí, sumada a otras métricas, es muy sospechosa.

Diferencia de funciones (Function_count_diff):

- En qué se basa: En el número de funciones declaradas.
- Aporte: Si la cantidad de funciones es la misma, refuerza la sospecha de que la estructura lógica fue copiada.

Diferencia de tamaño de archivo (Length_file_diff):

- En qué se basa: En la longitud total del archivo.
- Aporte: Si dos archivos tienen una longitud casi idéntica, es un fuerte indicador de duplicidad.

Complejidad ciclomática (Cyclomatic_complexity_diff):

- En qué se basa: En la complejidad lógica (cuántos caminos posibles toma el código).
- Cómo funciona: Calcula el número de decisiones (if, while, for) en ambos archivos.
- Aporte: Es difícil de ocultar. Si dos códigos resuelven lo mismo con la misma complejidad estructural, es probable que uno sea derivado del otro.

La combinación de métricas de correlación con métricas de divergencia permite al modelo mantener una alta precisión. Mientras que las similitudes de tokens y Levenshtein capturan la copia evidente, la Complejidad Ciclomática actúa como filtro que evita los falsos positivos al confirmar que la estructura es, efectivamente, la misma.

4.0 Preprocesado de los datos

Al revisar las features implementadas en el repositorio de código, aunque algunas son útiles, decidimos retomar exclusivamente el dataset de los códigos de Python. En los features iniciales, calculamos nuevamente todas las features de nuestro interés.

Por otra parte, seguíamos teniendo el problema del desequilibrio que representaba un gran sesgo para el modelo. Por lo que debíamos compensarlo para que quedara 50/50.

Adoptamos la estrategia que se refleja en PADBen Benchmark (2025) por Y. Zha, R. Min, and S. Sushmita [13] para expandir el dataset con códigos plagiados por diferentes IAs generativas y con distintos métodos de ofuscación como los presentados en Survey de métodos de plagio (2025) por Amirzhanov et al.[9]

Todas las features calculadas fueron escaladas para darles el mismo peso para evitar asignar un mayor impacto a algunas features para evitar sesgos al evaluar si es código se considera plagiado o no.

4.1 Corrección balance de clases

El desbalance de clases se corrigió con generación de código plagiado aplicando un random sampling de varios modelos:

- Deepseek v4 flash
- Gemini v2.5 flash
- Mistral Medium 3.1

Generando el resto de las 93 instancias plagiadas divididas en cada modelo, 31 instancias respectivamente.

El prompt utilizado fue:

4.2 Obtención de features

Una vez balanceado el dataset se trabajó en la obtención de features que alimentarán a los modelos, para esto se obtuvieron los features previamente mencionados:

- Ratio de comentarios (Comment_ratio_diff)
- Diferencia de funciones (Function_count_diff)
- Diferencia de tamaño de archivo (Length_file_diff):
- Complejidad ciclomática (Cyclomatic_complexity_diff):
- Levenshtein Similarity:

- Token Similarity:
- Árbol de Sintaxis Abstracta (AST Cosine Similarity):

Estás features son la variable de entrada, cada features son escalares relativos que se calculan comparando ambos códigos, con su respectiva implementación y métricas. Este enfoque en vez de hacer una implementación más clásica como un vector/matriz del texto procesado en tokens, esto nos permite tener una variedad más amplia de diferentes características que miden la similaridad entre código plagiado y no plagiado de acuerdo a la feature.

1196

4.3 Split de datos

Una vez con estos datos escalados, se le hace un split 80/20 (80% Train y 20% Test):

Porcentaje		Fin
100% →	80%	Entrenamiento
	20%	Test

4.4 Set de validación

Se generaron dos dataset de validación con dos posibles escenarios, teniendo en cuenta el post y pre uso de inteligencia artificial:

- Pre-Inteligencia artificial: se obtuvo en github bajo licencia de MIT con fecha antes de 2020, buscando código de python para regresión lineal bajo las keywords *"linear regression"* *language:Python committer-date:<2020-01-01* para código no plagiado y de código plagiado bajo licencia de MIT con fecha después de 2024, buscando código de python para regresión lineal bajo las keywords *"linear regression"* *language:Python committer-date:>2024-01-01* de diversos autores[17-32].
- Post-Inteligencia artificial: Se generó código a partir de nuestras instancias no plagiadas con 3 modelos de inteligencia artificial diferentes:
 - Deepseek v4 flash
 - Gemini v2.5 flash
 - Mistral Medium 3.1

El prompt usado fue: *"Reorder the code execution, modify variable names, function names, but keep the logic identical. Only return the modified code, with no explanations or comments. Do not use any formatting (no markdown backticks)."*

5.0 Modelado

5.1 Regresión logística

5.1.1 Descripción

Como referencia base de comparación, se eligió implementar un modelo de Regresión Logística utilizando las mismas características numéricas extraídas que se usaron en los modelos de redes neuronales. Este modelo proporciona un punto de referencia simple, interpretable y ligero para evaluar si modelos más complejos ofrecen una mejora significativa.

Este modelo no pretende obtener los mejores resultados, sino servir como punto de referencia para compararlo con otros modelos o arquitecturas, y discutir si modelos más complejos, como las redes neuronales, están justificados para este proyecto.

5.2 Multi Layer Perceptron Version 1

5.2.1 Descripción

El modelo neuronal diseñado para la detección de plagio tiene una arquitectura simple y enfocada en clasificación binaria. Su objetivo es analizar un conjunto de características numéricas previamente extraídas del código y determinar si un archivo corresponde a un código **plagiado** o **no plagiado**.

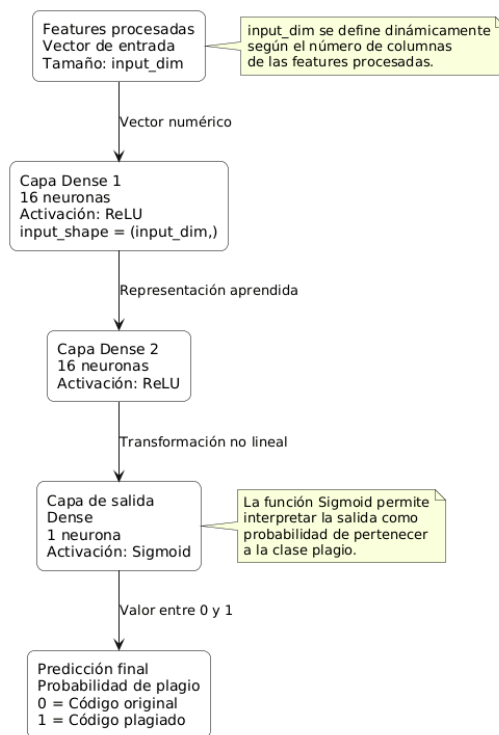
La entrada del modelo está definida por la variable `input_dim`, la cual representa el número total de características procesadas. Es decir, `input_dim` toma el valor correspondiente a la cantidad de columnas del conjunto de datos utilizado para entrenar el modelo.

5.2.2 Configuración y parámetros

Parámetro	Valor
Tipo de modelo	Red neuronal MLP
Tipo de problema	Clasificación binaria
Capa de entrada	Dense
Neuronas en capa de entrada	16
Activación capa de entrada	ReLU
Capa oculta	Dense
Neuronas en capa oculta	16
Activación capa oculta	ReLU
Capa de salida	Dense

Activación capa de salida	Sigmoid
Optimizador	Adam
Función de pérdida	binary_crossentropy
Métrica principal	Precision
Épocas	epochs = 50
Threshold	0.05
Batch	batch_size = 5

5.2.3 Arquitectura



5.3 Multi Layer Perceptron Version 2

5.3.1 Descripción

Este modelo solo representa una mejora respecto a la versión anterior. No hubo cambios en la arquitectura, por lo tanto se mantiene exactamente igual. Sin embargo se hizo un ajuste al parámetro al umbral (“*threshold*”), el cual se aborda con más detalle en el siguiente apartado.

5.3.2 Cambios y/o mejoras

Como se mencionó previamente, el único ajuste para esta versión fue un cambio en el umbral o “threshold” del modelo. Con este cambio se espera tener un modelo más conservativo o más “exigente” respecto a la clasificación si un fragmento de código es plagio o no.

Parámetro	Versión 1	Versión 2
Threshold	0.5	0.8

5.3 Multi Layer Perceptron Version 3

5.3.1 Descripción

La versión actual del modelo integra GraphCodeBERT en la cabeza del modelo versión 2 para realizar un análisis semántico de los códigos python. Este modelo pre-entrenado ha sido expuesto a vastos corpus de código en seis lenguajes de programación: Python, Java, JavaScript, PHP, Ruby y GO.

Después de que la cabeza haya procesado los datos, se le pasa al modelo versión 2 un coeficiente que indica la similitud semántica sacada por CodeBERT (**mrt_oast_cosine_similarity**). CodeBERT ayuda al MLP a identificar si la **lógica estructural** fue copiada, incluso si el código fue ofuscado o modificado por una IA

CodeBERT funciona como un encoder para transformar las secuencias de los nodos del árbol AST en vector numéricos (Embeddings). Primero generando un AST que es un Abstract Syntax Tree pero para evitar sesgos y ruidos innecesarios, el sistema implementa un **OAST (Optimized Abstract Syntax Tree)**. Este proceso consiste en omitir deliberadamente nodos relacionados con importaciones, namespaces y librerías, ya que estos elementos comunes inducen a falsos positivos en la detección de plagio. Al eliminar esto. El modelo se centra exclusivamente en el código.

Una vez obtenido el OAST, el sistema utiliza dos métodos de recorrido para capturar la esencia del código:

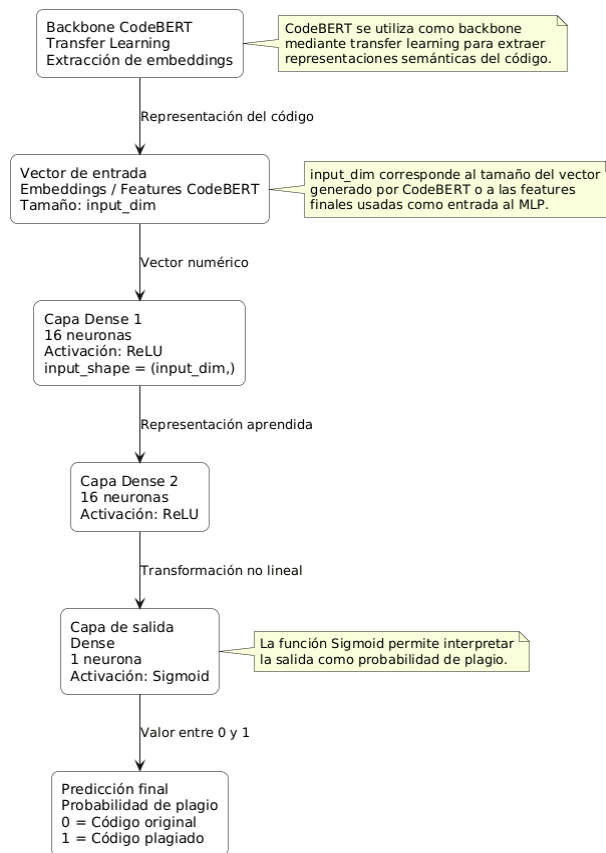
- **Recorridos Estructurales:** Se generan secuencias mediante recorridos Pre-order y Post-order del árbol optimizado. Esto permite capturar tanto la jerarquía general como la relación entre los terminales (variables y operadores). Ambas siendo métricas complementarias una de la otra. ya que si cambia la estructura o jerarquía general, no debería de afectar a la relación entre terminales si se quiere obtener un mismo resultado.
- **Resistencia a la ofuscación cambiando la estructura:** Aunque se altere el orden de ciertas estructuras, la secuencia de terminales y la lógica funcional mantienen una firma constante.
- **Embeddings y Representación:** Las secuencias se tokenizan y se procesan a través de CodeBERT para extraer vectores numéricos (embeddings). Se promedian los estados ocultos de ambas secuencias para consolidar un **vector de 768 dimensiones**, el cual actúa como una huella digital del archivo.

El modelo final de clasificación (MLP) no procesa el código fuente en bruto, sino que opera sobre una **matriz de características** pre-procesadas. La salida de CodeBert se traduce como una métrica de similitudes coseno denominada `mrt_oast_cosine_similarity` éste coeficiente representa una señal importante para el MLP en su evaluación ya que le permite discernir si la estructura lógica del código ha sido copiada o parafraseada, incluso si el código fue generado u ofuscado por herramientas generativas de IA

5.3.2 Cambios y/o mejoras

- Se sustituyó el Árbol de Sintaxis Abstracta (AST) convencional por una versión optimizada (OAST) que filtra automáticamente librerías e importaciones, reduciendo el ruido en la comparación de códigos.
- Se reemplazó el análisis puramente estadístico por un modelo de lenguaje pre-entrenado capaz de generar *embeddings* vectoriales de 768 dimensiones, lo que permite detectar similitudes semánticas incluso tras procesos de ofuscación.
- La inclusión de CodeBert ayuda a identificar patrones comunes en códigos generados por modelos como Gemini y DeepSeek, elevando la precisión en la detección de "paráfrasis de código".

5.3.4 Arquitectura



6.0 Resultados obtenidos

Tras entrenar el modelos, se le dió parte del dataset apartado para el test de que tan bien había aprendido y se obtuvieron lo siguientes resultados:

6.1 Métricas de evaluación

La métrica principal para evaluar este proyecto es la **precisión**. Dado que esta herramienta funciona como un diagnóstico auxiliar en la autoría de código, el éxito del modelo se define por su capacidad de asegurar que, cuando identifica un código como plagiado, **verdaderamente lo sea**. Aunque se reporten otras métricas para obtener una visión general, la precisión será el eje central para la toma de decisiones y mejora del modelo.

Precisión (Métrica Primaria):

La precisión mide la exactitud de las predicciones positivas. En este proyecto, una precisión alta es fundamental para evitar el fenómeno de las falsas alarmas.

- **Justificación:** Si la precisión es baja, el sistema marcará códigos legítimos como plagiados con frecuencia. Esto resta credibilidad a la herramienta y genera una carga de trabajo innecesaria (o incluso injusticias) al tener que desmentir detecciones erróneas.

- **Impacto:** Preferimos un modelo conservador que sólo actúa cuando la evidencia de plagio sea sólida. Al ser una herramienta de apoyo y no una decisión final, necesitamos que su diagnóstico sea confiable y certero para no entorpecer el flujo de trabajo del usuario o la empresa.

Recall:

El Recall mide la capacidad de capturar todos los casos de plagio existentes. En este enfoque, aceptamos que el Recall pueda quedar en segundo plano. **El Impacto de un Recall bajo** Un modelo con alta precisión pero menor recall puede ser más permisivo, dejando pasar algunos casos de plagio (Falsos Negativos). Sin embargo, bajo el contexto de este proyecto, se asume que es preferible que algunos casos se filtren a cambio de garantizar que ninguna alerta emitida sea un error. No buscamos una vigilancia absoluta que sacrifique la confiabilidad de cada reporte.

F1-Score:

El F1-Score busca un equilibrio entre Precisión y Recall. Debido a que nuestro objetivo no es el equilibrio, sino la maximización de la tasa de aciertos positivos (Precisión), el F1-Score no se considera una métrica determinante. Un F1-Score moderado será aceptable siempre y cuando la precisión se mantenga en niveles óptimos.

6.2 Resultados

Modelo	Recall	Precision	F1-Score
Regresión logística	0.84	0.67	0.74
MLP Version 1	0.82	0.95	0.88
MLP Version 2	0.59	0.93	0.72
MLP Version 3	0.73	0.94	0.82

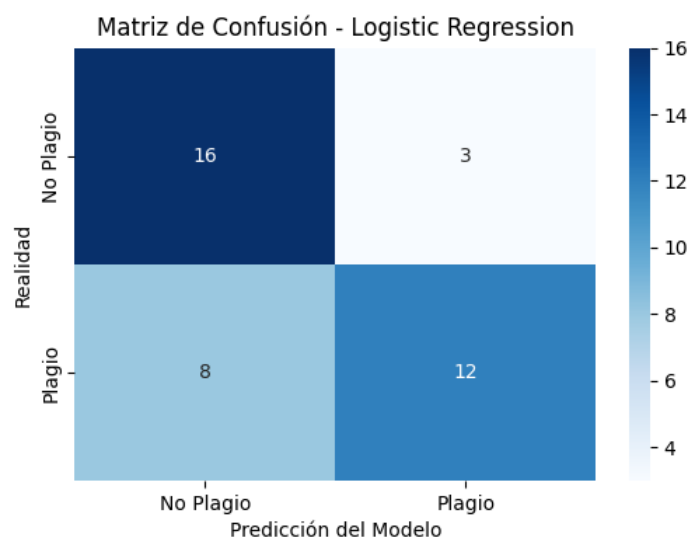


Figura 4.0: Matriz de confusión del modelo de regresión logística

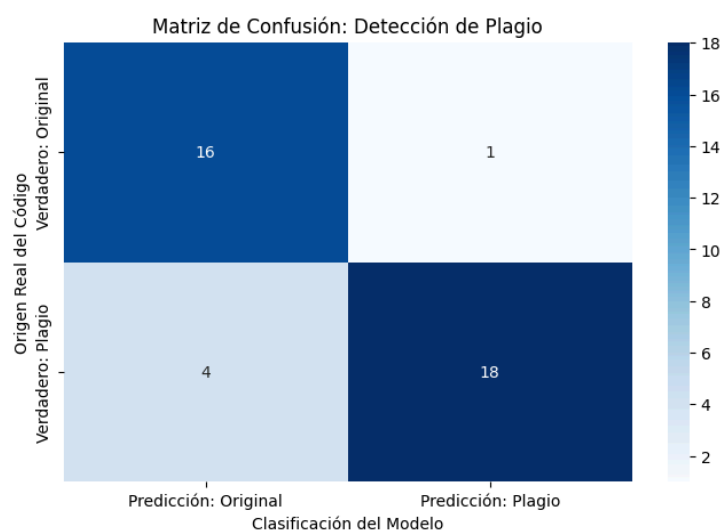


Figura 5.0: Matriz de confusión del modelo MLP con umbral de detección de 0.5

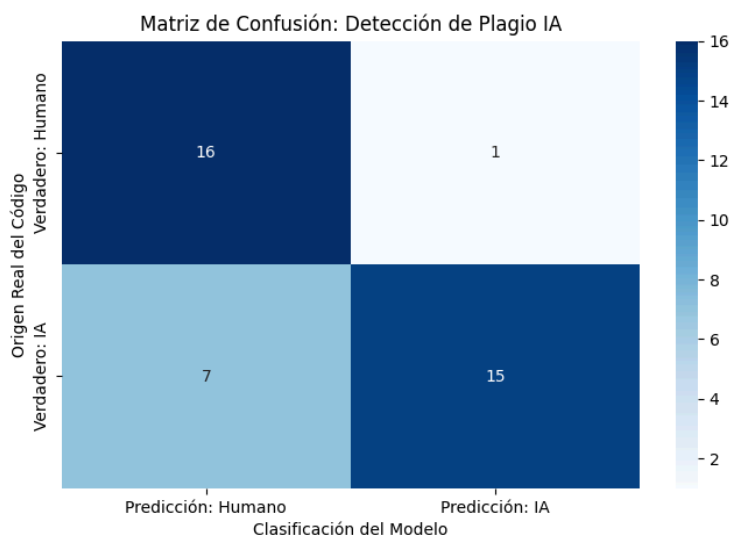


Figura 6.0: Matriz de confusión del modelo MLP con umbral de detección de 0.8

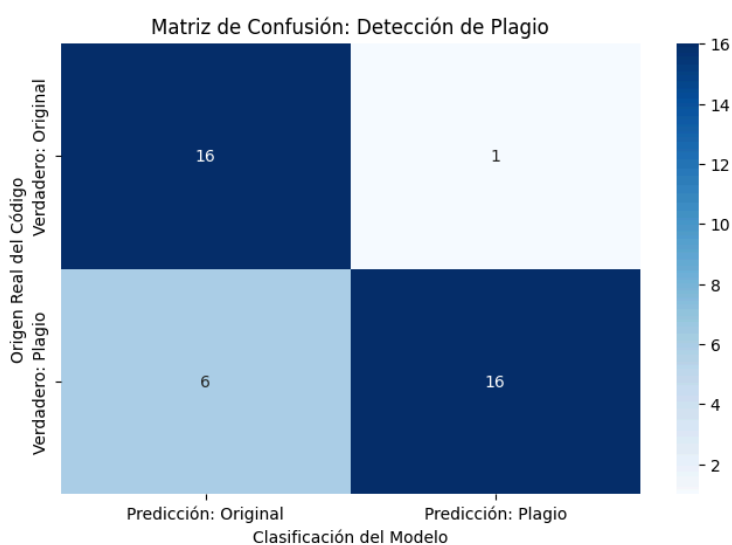


Figura 7.0: Matriz de confusión del modelo MLP con umbral de detección de 0.8 y con el uso de GraphCodeBERT en la cabeza del modelo

Neural Network Evaluation:				
	precision	recall	f1-score	support
Original - 0	0.88	0.82	0.85	17
Plagio - 1	0.87	0.91	0.89	22
accuracy			0.87	39
macro avg	0.87	0.87	0.87	39
weighted avg	0.87	0.87	0.87	39
AUC-ROC Score: 0.9572				

6.3 Mejoras

- **Ajuste de los hiper parámetros (umbral de detección)**
 - El Umbral de detección es una tolerancia al momento de clasificar los datos. normalmente está en el 0.5. como default. Es decir, que cuando un resultado es 0.5 o más adelante se considera 1 (Plagio) y abajo de eso es 0 (Legítimo). En este caso se puede ajustar el umbral como a 0.7 o 0.8 (haciendo que el la detección sea más estricta. Por debajo de eso, todo es plagio. Esto llevaría al MLP a su versión 2
- **Ajuste de la arquitectura de los datos.**
 - Destilación de modelos pre-entrenados como codeBERT para ayudar a la detección de código plagiado

7.0 Ventajas y Desventajas del Modelos

7.1 Regresión logística

7.1.1 Ventajas

- Modelo simple, rápido e interpretable.
- Sirve como línea base para comparar modelos más complejos.
- Requiere pocos recursos computacionales.
- Permite observar qué tan útiles son las features extraídas sin usar una arquitectura compleja.

7.1.2 Desventajas

- Tiene menor precisión que los modelos MLP.
- No captura relaciones no lineales complejas entre las features.
- Puede ser limitada ante casos de plagio con ofuscación o modificaciones estructurales.
- Su desempeño depende mucho de la calidad de las características manuales.

7.2 MLP Version 1

7.2.1 Ventajas

- Presenta mejor desempeño que la regresión logística.
- Captura relaciones no lineales entre las características.
- Mantiene una arquitectura ligera y fácil de entrenar.
- Obtuvo una precisión alta, lo cual es importante para reducir los falsos positivos.

7.2.2 Desventajas

- Puede ser sensible al tamaño reducido del dataset.
- Su arquitectura es simple y puede no capturar patrones más complejos.
- Tiene menor interpretabilidad que la regresión logística.
- Puede sobre ajustarse si los datos no son suficientemente variados.

7.3 MLP Version 2

7.3.1 Ventajas

- Ajusta el threshold para hacer el modelo más estricto.
- Reduce el riesgo de clasificar código legítimo como plagio.
- Mantiene la misma arquitectura ligera de la versión 1.
- Es útil cuando se prioriza la precisión sobre el recall.

7.3.2 Desventajas

- Al subir el threshold, puede dejar pasar más casos reales de plagio.
- Presenta menor recall que la versión 1.
- La mejora se basa principalmente en un ajuste de umbral, no en un cambio arquitectónico.
- Puede ser demasiado conservador dependiendo del contexto de uso.

7.4 MLP Version 3

7.4.1 Ventajas

- Eficiencia en el Procesamiento y Entrenamiento: Rápida iteración sobre los dataset.
- Optimización de Recursos (Arquitectura Liger): El modelo se caracteriza por su ligereza estructural, lo que se traduce en un consumo reducido de memoria RAM para entrenar.
- Versatilidad y Ajuste Paramétrico: La configuración del modelo permite una alta capacidad de ajuste (*fine-tuning*). Esto facilita la calibración de los umbrales de decisión para adaptarse a diferentes lenguajes de programación o niveles de exigencia académica, permitiendo que la herramienta evolucione según las necesidades específicas de la institución o empresa.
- Interpretatividad y Transparencia Operativa: La correlación directa entre las características de entrada (como el conteo de variables o la longitud del archivo) y la salida es de más fácil interpretación.
- Especialización en python: El estado del arte usualmente es para lenguajes como java o C++.

7.4.2 Desventajas

- Naturaleza del Diagnóstico: El modelo está concebido estrictamente como una herramienta de análisis auxiliar para la supervisión de autoría. Sus resultados proporcionan criterios de juicio, como un soporte técnico ante hallazgos sospechosos previos.
- Restricción de Entrada: El modelo sólo permite la comparación de archivos únicos. No soporta la evaluación de proyectos cuya lógica se encuentre distribuida en múltiples archivos.
- Sesgo por sencillez: En código muy corto, existe un sesgo que marca como plagio el código. Y es que por la naturaleza de python, al ser un lenguaje dinámico, es mucho más difícil determinar qué es plagio.
- Alcance en la Taxonomía de Plagio (L1, L2 y potencialmente L3): El modelo presenta una delimitación técnica basada en la complejidad de las ofuscaciones aplicadas al código fuente. El sistema está diseñado para detección de similitudes que van desde la copia literal hasta la reestructuración funcional básica
 - Niveles Cubiertos: Capacidad de detectar cambios en las variables (L1), cambios del flujo del código (L2), control de flujo y renombramiento de las variables (L3).
 - Niveles Excluidos: El modelo no está diseñado para procesar plagio a partir del nivel L4 (reemplazo de estructuras de control por equivalentes), L5 (cambio total de implementación) ni L6 (cambio de paradigma de programación).

7.4.3 Áreas de oportunidad

- Evaluar el aporte real de GraphCodeBERT mediante un estudio de ablación.
- Optimizar el threshold de manera automática.

- Aplicar validación cruzada para obtener resultados más robustos.
- Explorar arquitecturas tipo Siamese Network para comparar pares de código directamente.

8.0 Conclusiones generales

Tras los resultados obtenidos, se concluye que los modelos implementados lograron establecer una base funcional para la detección de plagio en código Python. La regresión logística sirvió como referencia inicial, mientras que las versiones del MLP mostraron un mejor desempeño al aprovechar características léxicas, estructurales y semánticas. En particular, la versión 3 incorporó GraphCodeBERT, lo que permitió analizar similitudes más profundas entre códigos modificados u ofuscados.

Sin embargo, el dataset inicial presenta limitaciones importantes. Debido a que muchos estudiantes resolvían la misma actividad, es natural que existieran soluciones similares aunque no necesariamente plagiadas. Esto genera ambigüedad y puede afectar la capacidad del modelo para diferenciar entre similitud legítima y plagio real.

Como trabajo futuro, se propone mejorar el dataset con más ejemplos variados, aplicar validación cruzada para obtener métricas más confiables, ajustar hiperparámetros como el threshold y explorar cambios en la arquitectura del modelo. Aunque se incorporaron elementos del estado del arte, todavía existen áreas de oportunidad para mejorar la generalización, precisión e interpretabilidad del sistema.

En general, el modelo debe entenderse como una herramienta auxiliar de análisis, no como una decisión final sobre plagio. Su utilidad principal es aportar evidencia técnica que pueda ser revisada junto con criterio humano.

Citas

- [1] T. Tokita, “Vibe Coding Works. Until It Doesn’t. What the Vercel Breach Should Teach Every Developer.” DEV Community, abril de 2026. Consultado: el 28 de abril de 2026. [En línea]. Disponible en:
<https://dev.to/tomtokita/vibe-coding-works-until-it-doesnt-what-the-vercel-breach-should-teach-every-developer-386k>
- [2] L. C. Ming, “Replit CEO apologizes after AI coding tool wipes company’s database”. Business Insider, julio de 2025. [En línea]. Disponible en:
<https://www.businessinsider.com/replit-ceo-apologizes-ai-coding-tool-delete-company-database-2025-7?op=1>
- [3] T. Carter, “A startup says Cursor’s AI agent deleted its production database”. Business Insider, abril de 2026. Consultado: el 28 de abril de 2026. [En línea]. Disponible en:
<https://www.businessinsider.com/pocketos-cursor-ai-agent-deleted-production-database-startup-railway-2026-4>
- [4] M. A. El-Rashidy, R. G. Mohamed, N. A. El-Fishawy, y M. A. Shouman, “Reliable plagiarism detection system based on deep learning approaches”, *Neural Comput. Appl.*, jun. 2022, doi: 10.1007/s00521-022-07486-w.
- [5] O. Kamat, T. Ghosh, K. J. A. V, y R. P, “Plagiarism Detection Using Machine Learning”. arXiv.org, 2024. [En línea]. Disponible en: <https://arxiv.org/abs/2412.06241>
- [6] M. Zeng, “Research on AI-Generated Text Detection Based on Machine Learning Models”, *Trans. Comput. Sci. Intell. Syst. Res.*, vol. 7, pp. 229–233, nov. 2024, doi: 10.62051/8k1jga32
- [7] I. H. Sadhin, T. Hassan, y A. M. Nayim, “Plagiarism Detection Using Artificial Intelligence”, *Int. J. Comput. Inf. Syst. IJCIS*, vol. 05, pp. 102–108, jun. 2024, doi: 10.29040/ijcis.v5i2.170
- [8] A. M. Elkhatat, K. Elsaid, y S. Al-Meer, “Evaluating the efficacy of AI content detection tools in differentiating between human and AI-generated text”, *Int. J. Educ. Integr.*, vol. 19, sep. 2023, doi: 10.1007/s40979-023-00140-5.
- [9] A. Amirzhanov, C. Turan, y A. Makhmutova, “Plagiarism types and detection methods: a systematic survey of algorithms in text analysis”, *Front. Comput. Sci.*, vol. 7, mar. 2025, doi: 10.3389/fcomp.2025.1504725.
- [10] H. Suh, M. Tafreshipour, J. Li, A. Bhattiprolu, y I. Ahmed, “An Empirical Study on Automatically Detecting AI-Generated Source Code: How Far Are We?” arXiv.org, 2024. [En línea]. Disponible en: <https://arxiv.org/abs/2411.04299>
- [11] H. Guo, S. Cheng, K. Zhang, G. Shen, y X. Zhang, “CodeMirage: A Multi-Lingual Benchmark for Detecting AI-Generated and Paraphrased Source Code from Production-Level LLMs”. arXiv.org, 2025. Consultado: el 28 de abril de 2026. [En línea]. Disponible en:
<https://arxiv.org/abs/2506.11059>
- [12] C. Mao, C. Vondrick, H. Wang, y J. Yang, “Raidar: geneRative AI Detection viA Rewriting”. arXiv.org, 2024. [En línea]. Disponible en: <https://arxiv.org/abs/2401.12970>

- [13] Y. Zha, R. Min, y S. Sushmita, “PADBen: A Comprehensive Benchmark for Evaluating AI Text Detectors Against Paraphrase Attacks”. arXiv.org, 2025. Consultado: el 28 de abril de 2026. [En línea]. Disponible en: <https://arxiv.org/abs/2511.00416>
- [14] S. Bashir, “Using pseudo-AI submissions for detecting AI-generated code”, *Front. Comput. Sci.*, vol. 7, may 2025, doi: 10.3389/fcomp.2025.1549761.
- [15] A. Amigud, J. Arnedo-Moreno, T. Daradoumis, y A.-E. Guerrero-Roldan, “An integrative review of security and integrity strategies in an academic environment: Current understanding and emerging perspectives”, *Comput. Secur.*, vol. 76, pp. 50–70, 2018, doi: <https://doi.org/10.1016/j.cose.2018.02.021>.
- [16] ehsankhani, “GitHub - ehsankhani/cheating_detector: cheating detection system with python for the code home works to be detected if thay have cheating or not and generate reoprts”. GitHub, 2025. Consultado: el 28 de abril de 2026. [En línea]. Disponible en: https://github.com/ehsankhani/cheating_detector
- [17] sarvasvkulpati, “LinearRegression.py,” *LinearRegression*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/sarvasvkulpati/LinearRegression/refs/heads/master/LinearRegression.py>.
- [18] barulalithb, “Linear Regression from scratch.py,” *Linear-Regression-from-Scratch*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/barulalithb/Linear-Regression-from-Scratch/refs/heads/master/LinearRegressionfromscratch.py>. [Accessed: Apr. 28, 2026].
- [19] capt-calculator, “linreg_scratch.py,” *linear_regression_from_scratch_py*, GitHub. [Online]. Available: https://raw.githubusercontent.com/capt-calculator/linear_regression_from_scratch_py/refs/heads/master/linreg_scratch.py. [Accessed: Apr. 28, 2026].
- [20] LarsNeR, “LinearRegression.py,” *LinearRegression*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/LarsNeR/LinearRegression/refs/heads/master/LinearRegression.py>. [Accessed: Apr. 29, 2026].
- [21] perborgen, “linear.py,” *KeepStraight*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/perborgen/KeepStraight/refs/heads/master/linear.py>. [Accessed: Apr. 28, 2026].
- [22] Jamaliela, “Linear_Regression.py,” *LinearRegression_fromScratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/Jamaliela/LinearRegression_fromScratch/refs/heads/master/LinearRegression.py. [Accessed: Apr. 28, 2026].
- [23] ettore34, “test.py,” *linearRegressionFromScratch-*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/ettore34/linearRegressionFromScratch-/refs/heads/master/test.py>. [Accessed: Apr. 28, 2026].
- [24] 0xHadyy, “Linear_Regression_Implementation.py,” *Linear-Regression-Scratch*, GitHub. [Online]. Available:

https://raw.githubusercontent.com/0xHady/Linear-Regression-Scratch/refs/heads/master/Linear_Regression_Implementation.py. [Accessed: Apr. 28, 2026].

[25] l0stinfr0st, “LinearRegression.py,” *LinearRegressionScratch*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/l0stinfr0st/LinearRegressionScratch/refs/heads/main/LinearRegression.py>. [Accessed: Apr. 28, 2026].

[26] sara-iskandar, “linear_regression.py,” *linear-regression-scratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/sara-iskandar/linear-regression-scratch/refs/heads/main/linear_regression.py. [Accessed: Apr. 28, 2026].

[27] NishantSagar12345, “linear_regression_from_scratch.py,” *LinearRegressionScratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/NishantSagar12345/LinearRegressionScratch/refs/heads/main/linear_regression_from_scratch.py. [Accessed: Apr. 28, 2026].

[28] VasuML07, “Linear.py,” *LinearRegressionfromScratch*, GitHub. [Online]. Available: <https://raw.githubusercontent.com/VasuML07/LinearRegressionfromScratch/refs/heads/main/Linear.py>. [Accessed: Apr. 28, 2026].

[29] OmarWill1, “linearRegression.py,” *LinearRegression_Scratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/OmarWill1/LinearRegression_Scratch/refs/heads/main/linearRegression.py. [Accessed: Apr. 28, 2026].

[30] ArayikKarapetyan, “linear_regression.py,” *linear-regression-scratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/ArayikKarapetyan/linear-regression-scratch/refs/heads/main/src/linear_regression.py. [Accessed: Apr. 28, 2026].

[31] NepSauce, “linear_regression.py,” *linear-regression-scratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/NepSauce/linear-regression-scratch/refs/heads/main/linear_regression.py. [Accessed: Apr. 28, 2026].

[32] oscarbiggins, “linear_regression_scratch.py,” *Linear_Regression_Scratch*, GitHub. [Online]. Available: https://raw.githubusercontent.com/oscarbiggins/Linear_Regression_Scratch/refs/heads/main/linear_regression_scratch.py. [Accessed: Apr. 28, 2026].

[33] T. Yu, L. Yuan, L. Lin, and H. He, “A multiple representation transformer with optimized abstract syntax tree for efficient code clone detection,” *ACM DL DIGITAL LIBRARY*, pp. 281–293, Apr. 2025, doi: 10.1109/icse55347.2025.00050.

[34] DeepWiki, “GraphCodeBERT,” *DeepWiki*, Apr. 23, 2025. <https://deepwiki.com/microsoft/CodeBERT/2-graphcodebert>

